Managing Secrets and Configuration in CI/CD

1 Comment / By Dot Net Tutorials / April 6, 2026

Back to: Microservices using ASP.NET Core Web API Tutorials

Managing Secrets and Configuration in CI/CD

In the previous chapter, we built a CI pipeline that automatically builds, tests, and publishes our application. Now comes a very important real-world topic: **security and configuration management**. In real projects, we never store sensitive information like database passwords, API keys, or deployment credentials directly in code or YAML files. Instead, we use secure mechanisms like GitHub Secrets and environment-based configuration.

What We Will Do in This Chapter?

By the end of this chapter, we will do the following practical tasks:

- Add clean configuration sections to our ASP.NET Core Web API project.
- Keep non-sensitive settings in the appsettings.json file.
- Keep environment-specific values in appsettings.Development.json and appsettings.Production.json files.
- Create GitHub Variables for non-sensitive CI/CD values.
- Create GitHub Secrets for sensitive values.
- Create GitHub Environments such as Development and Production.
- Create GitHub Environments specific variables and secrets.
- Update the GitHub Actions workflow to read variables and secrets safely.

This gives us a clean foundation for deployment automation in the next chapter.

Step 1: First, Understand What Goes Where

Normal settings go in configuration files. Sensitive values go in secrets. Pipeline-only non-sensitive values go in variables. The simple rule:

- **appsettings.json** = Normal application configuration.
- **appsettings.Development.json** = Development-only values.
- **appsettings.Production.json** = Production-only values.
- **GitHub Variables** = Non-sensitive pipeline values.
- **GitHub Secrets** = Sensitive pipeline values such as passwords, tokens, API keys, and connection strings.
- **GitHub Environments** = Environment-specific secrets and variables.

Step 2: Add Clean Application Configuration to appsettings.json

Now, let us improve our project configuration. Open appsettings.json and update it like this:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
{
  "ApplicationSettings": {
    "ApplicationName": "Student Management API",
    "SupportEmail": "support@demoapp.com"
  },
  "ExternalServices": {
    "StudentPortalBaseUrl": "https://localhost:7001"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

This file should contain only safe, normal configuration values. It is fine to store items such as the application name, base URL, or support email here, as long as the values are not sensitive.

Step 3: Add Development-Specific Configuration

Now open **appsettings.Development.json** and add values that differ when running locally.

```
{
  "ApplicationSettings": {
    "ApplicationName": "Student Management API - Development",
    "SupportEmail": "devsupport@demoapp.com"
  },
  "ExternalServices": {
    "StudentPortalBaseUrl": "https://dev-studentportal.local"
  }
}
```

This file is used when the app runs in the Development environment. ASP.NET Core supports environment-specific configuration files, such as appsettings.Development.json, which override corresponding values in the main appsettings.json.

So now:

- appsettings.json gives the default value
- appsettings.Development.json overrides that value in Development

Step 4: Add Production-Specific Configuration

Now, create or update appsettings.Production.json like this:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
{
  "ApplicationSettings": {
    "ApplicationName": "Student Management API - Production",
    "SupportEmail": "support@studentmanagement.com"
  },
  "ExternalServices": {
    "StudentPortalBaseUrl": "https://studentmanagement.com"
  }
}
```

When the app runs in the production environment, these values can override the defaults. Development and Production should not always use the same values. That is why environment-specific files exist.

Step 5: Read Configuration Inside the API

Now, let us see that the configuration is actually working. When we run the app locally in Development, the endpoint will show Development values. Later, when the app runs in Production, the endpoint can show Production values instead. Open **HealthController.cs** and update it like this:

```
using Microsoft.AspNetCore.Mvc;
namespace StudentManagement.API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class HealthController : ControllerBase
    {
        private readonly IConfiguration _configuration;

        public HealthController(IConfiguration configuration)
        {
            _configuration = configuration;
        }

        [HttpGet]
        public IActionResult Get()
        {
            return Ok(new
            {
                Status = "Healthy",
                ApplicationName =
_configuration["ApplicationSettings:ApplicationName"],
                SupportEmail =
_configuration["ApplicationSettings:SupportEmail"],
                StudentPortalBaseUrl =
_configuration["ExternalServices:StudentPortalBaseUrl"]
            });
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
        }
    }
}
```

ASP.NET Core exposes configuration through IConfiguration, and configuration keys can be read using hierarchical paths like Section:ChildKey.

Step 6: Run the Project and Test the Configuration

Run the project from Visual Studio and call: https://localhost:xxxx/api/health

If the app is running in Development, you should see the Development values from appsettings.Development.json.

Step 7: Create a GitHub Repository Secret

Now, let us do a real GitHub step. Open your GitHub repository and go to:

- **Settings**
- **Secrets and variables**
- **Actions**
- **New repository secret**

Create a sample secret like this:

- Name: **DEMO_API_KEY**
- Value: **Any Sample Secret Value**
- Then click on the **Add Secret** button

GitHub repository secrets can be stored at the repository scope, and workflows can access them using the secrets context.

Step 8: Create a GitHub Repository Variable

Now, create one non-sensitive variable. In the same GitHub area:

- **Settings**
- **Secrets and variables**
- **Actions**
- **Variables**
- **New repository variable**

Create something like:

- Name: **APP_ENVIRONMENT**

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- Value: **Development**
- Then click on the Add Variable button

GitHub variables are meant for non-sensitive configuration values, and workflows can access them using the vars context.

Step 9: Use the Secret and Variable in the Workflow

Creating a variable or secret in GitHub only stores it in GitHub. It does not automatically become available inside every workflow step**.** GitHub says variables are accessed through the vars context, and secrets are only readable in a workflow when you explicitly include them.

Let's understand it with one simple analogy. Think of GitHub as a locker room.

- A **variable** or **secret** is like an item stored in a locker.
- Your **workflow** is like a worker entering the room.
- The worker does **not** automatically carry every item from every locker.
- You must explicitly say which item to take and where to use it.

Syntax: ${{ vars.APP_ENVIRONMENT }}** or **${{ secrets.DEMO_API_KEY }} in the workflow.

Example to use Variables in a workflow:

– name: Show non-sensitive variable

run: echo "Workflow environment is ${{ vars.APP_ENVIRONMENT }}"

Here, **APP_ENVIRONMENT** is a GitHub **variable**, so we access it with: **${{ vars.APP_ENVIRONMENT }}**

GitHub reads the value of APP_ENVIRONMENT and replaces it before the command runs. So, if you created this variable in GitHub:

- APP_ENVIRONMENT = Development

Then this line behaves like:

- run: echo "Workflow environment is Development."

So, the log shows: Workflow environment is Development

Example to use Secret in a workflow:

– name: Check secret availability

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

env:

DEMO_API_KEY: ${{ secrets.DEMO_API_KEY }}

run: |

if [-z “$DEMO_API_KEY”]; then

echo “Secret is missing.”

exit 1

fi

echo “Secret is available for workflow use.”

Code Explanation:

This is the part that needs the most clarity.

Step-A: secrets.DEMO_API_KEY

This part: **${{ secrets.DEMO_API_KEY }}** means read the stored secret value from GitHub.

Step-B: Why put it inside env?

env:
DEMO_API_KEY: ${{ secrets.DEMO_API_KEY }}

This means create an environment variable named **DEMO_API_KEY** for this step, and fill it with the GitHub secret value.

GitHub allows secrets to be passed to a step as an environment variable using env. This is why inside the shell script we use **$DEMO_API_KEY** and not **${{ secrets.DEMO_API_KEY }}**

So, the flow is:

1. GitHub reads the secret
2. GitHub puts it into the step’s environment variable
3. The shell script uses that environment variable

Step-C: What does this check do?

if [-z “$DEMO_API_KEY”]; then

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

echo “Secret is missing.”

exit 1
fi

Here:

-z means is this string empty?

So, this logic says:

- If the secret value is empty, print “Secret is missing”
- Then stop the workflow with exit 1

Because GitHub returns an empty string when a secret is missing, this is a safe way to verify that the secret exists.

Step 10: Create GitHub Environments

Now, we will move from repository-wide or global settings to environment-specific settings.

In GitHub, go to:

- **Settings**
- **Environments**
- **New environment**

Create:

- Development
- Production

GitHub Environments are used to group environment-specific deployment controls, such as environment secrets and variables.

Why This Matters?

Repository-level secrets and variables apply broadly. Environment-level secrets and variables let you say:

- Development uses one value
- Production uses another value

That is exactly what we want in real CI/CD.

Step 11: Add Environment-Specific Secret and Variable

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Now, inside the **Development** environment, add:

- Secret: **DEPLOY_API_KEY = dev-deploy-key-123**
- Variable: **TARGET_SERVER_NAME = Dev-IIS-Server**

Inside the **Production** environment, add:

- Secret: **DEPLOY_API_KEY = prod-deploy-key-456**
- Variable: **TARGET_SERVER_NAME = Prod-IIS-Server**

Step 12: Connect a Job to an Environment

Now update the workflow job like this:

jobs:
build-test-publish:
name: Build, Test, and Publish

runs-on: ubuntu-latest

environment:
name: Development

deployment: false

This tells GitHub:

- Run this job using the **Development environment**
- Apply any rules configured on that environment
- Make the **Development environment secrets and variables** available to this job
- But do **not** treat it as a real deployment record because deployment: false is set. This is specifically useful for CI or testing jobs that need environment secrets without actually deploying.

Step 13: Final Workflow Example

The following version uses all four clearly:

- **Repository variable**: APP_ENVIRONMENT
- **Repository secret**: DEMO_API_KEY
- **Environment variable**: TARGET_SERVER_NAME
- **Environment secret**: DEPLOY_API_KEY
- **Job attached to the Development environment**

Please update the **ci.yml** as follows.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```yaml
name: StudentManagement API CI

on:
  # Run CI when code is pushed to main or any feature branch
  push:
    branches:
      - main
      - feature/**

  # Run CI when a pull request is created or updated for main
  pull_request:
    branches:
      - main

jobs:
  build-test-publish:
    name: Build, Test, and Publish
    runs-on: ubuntu-latest

    # Attach this job to the Development environment
    # So, it can use environment-level variables and secrets
    environment:
      name: Development
      deployment: false

    steps:
      # Step 1: Download the repository code
      - name: Checkout source code
        uses: actions/checkout@v5

      # Step 2: Install .NET 8 SDK
      - name: Setup .NET 8 SDK
        uses: actions/setup-dotnet@v5
        with:
          dotnet-version: '8.0.x'

      # Step 3: Show repository-level variable
      - name: Show repository variable
        run: echo "Workflow environment is ${{ vars.APP_ENVIRONMENT }}"

      # Step 4: Show environment-level variable
      - name: Show environment variable
        run: echo "Target server is ${{ vars.TARGET_SERVER_NAME }}"

      # Step 5: Check repository-level secret
      - name: Check repository secret
        env:
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
          DEMO_API_KEY: ${{ secrets.DEMO_API_KEY }}
        run: |
          if [ -z "$DEMO_API_KEY" ]; then
            echo "Repository secret DEMO_API_KEY is missing."
            exit 1
          fi
          echo "Repository secret is available."

      # Step 6: Check environment-level secret
      - name: Check environment secret
        env:
          DEPLOY_API_KEY: ${{ secrets.DEPLOY_API_KEY }}
        run: |
          if [ -z "$DEPLOY_API_KEY" ]; then
            echo "Environment secret DEPLOY_API_KEY is missing."
            exit 1
          fi
          echo "Environment secret is available."

      # Step 7: Restore NuGet packages
      - name: Restore dependencies
        run: dotnet restore StudentManagement.API.sln

      # Step 8: Build the solution in Release mode
      - name: Build solution
        run: dotnet build StudentManagement.API.sln --configuration Release --no-
restore

      # Step 9: Run unit tests
      - name: Run tests
        run: dotnet test StudentManagement.API.sln --configuration Release --no-
build --verbosity normal

      # Step 10: Publish the API project
      - name: Publish Web API
        run: dotnet publish StudentManagement.API/StudentManagement.API.csproj --
configuration Release --output ./publish --no-build

      # Step 11: Store the publish output as an artifact
      - name: Upload publish artifact
        uses: actions/upload-artifact@v6
        with:
          name: studentmanagement-api-publish
          path: ./publish
```

Conclusion

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

In this chapter, we prepared our **StudentManagement.API** project to improve CI/CD safety by separating normal configuration from sensitive values. We stored normal application settings in appsettings.json, used environment-specific JSON files for Development and Production, created GitHub variables for non-sensitive pipeline values, created GitHub secrets for sensitive values, and introduced GitHub Environments for environment-level control.

Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent | Do Not Sell My Information

Explore our developer-friendly HTML to PDF API | Printed using PDFCrowd | HTML to PDF